

HIGH PERFORMANCE COMPUTING FOR SCIENCE AND ENGINEERING

Harvard University
CS 2050

Lecture 15

March 24, 2026

UPDATE ON CLUSTER



HARVARD UNIVERSITY
Information Technology

OnDemand provides an integrated, single access point for all of your HPC resources.

Latest News for HUIT Open OnDemand

Where to go for help

For a basic overview of the HUIT implementation of Open OnDemand, visit the [HUIT Open OnDemand documentation page](#). If your question can't be answered there, check with your instructor or other course staff, and write to ithelp@harvard.edu if you can't figure it out together.

Regular patching

The servers running HUIT Open OnDemand require regular patching, which is scheduled to occur on Monday mornings at 4am. During this time, the platform will be unavailable. Patches usually take less than 1 hour, but could take longer.

QUIZ 7

[Account](#)[Dashboard](#)[Courses](#)[Calendar](#)[Inbox](#)[History](#)[Help](#)

[COMPSCI 2050](#) > [Quizzes](#) > [Quiz 7](#)

2025-2026 Spring

[Home](#)[Announcements](#)[Grades](#)[Course Calendar](#)[Q & A](#)[Zoom](#)[DCE Class Recordings](#)[Modules](#)[Quizzes](#)[Gradescope](#)[Files](#)[People](#)[Secure Exam Proctor](#)

Quiz 7

Due Mar 24 at 11:59pm	Points 3	Questions 3
Available Mar 23 at 12am - Jul 1 at 11:59pm	Time Limit 60 Minutes	

Instructions

The quizzes are designed to take no more than 30 minutes, but one hour will be allowed.

Do not click into the quiz until you are ready to take it.

Take the Quiz

Copyright © 2013-2026 The President and Fellows of Harvard College | [Acceptable Use Policy](#) | [Accessibility](#) | [Digital Accessibility](#) | [Privacy Policy](#)

HOMeworks

[Security](#) [Insights](#) [Settings](#)

🔒 homework-2 Internal template

[Edit Pins](#) [Watch 0](#) [Fork 0](#)

[solution](#) [2 Branches](#) [0 Tags](#)

[Add file](#) [Code](#)

This branch is 1 commit ahead of [main](#).

[Contribute](#)

 wcw398

 solution. 2087015 · 18 hours ago 2 Commits

question-1	solution.	18 hours ago
question-2	solution.	18 hours ago
question-3	solution.	18 hours ago
question-4	solution.	18 hours ago
question-5	solution.	18 hours ago
question-6	solution.	18 hours ago
question-7	public release.	last month
.gitignore	public release.	last month
README.md	solution.	18 hours ago

📖 README

[Edit](#) [Menu](#)

Homework 2 Solution

You may find it useful to [compare this solution](#) with the original assignment.

 laz380 and Chuck Witt draft release. 31d14b3 · 2 minutes ago 1 Commits

 README.md draft release. 2 minutes ago

CS 2050 Final Project

Warning

This is a DRAFT of the final project specification, as of *March 5, 2026*. Please review and ask any clarification questions over the next two weeks. Based on feedback, we will refine and release the final specification at the end of the Spring Recess.

Overview

The final project requires you to apply the high-performance computing techniques learned in class to a computational task of your choice. The project will be completed **individually**. You will begin with a serial version of your algorithm, and then develop parallel versions using OpenMP, MPI, and CUDA. You will analyze its performance in scaling as well as with profiling tools. Finally, you will explore an additional topic beyond the core course material.

 **questions-and-answers** Public

 Edit Pins ▾

 Watch 0 ▾

 Fork 0 ▾

 Star 0 ▾

 main ▾  

🔍 Go to file t + <> Code ▾ About 

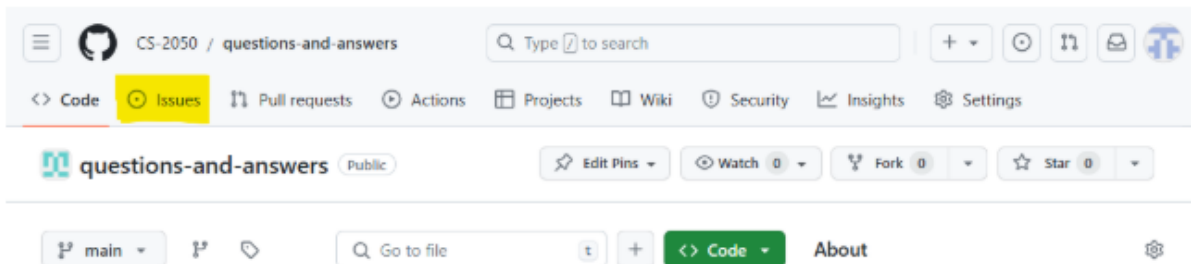
 **wcw398** update readme 4be70a2 · 17 hours ago 🕒 4 Commits

 README.md update readme 17 hours ago

 README

Questions and Answers

Please create issues in this repository to ask questions about the course.



About

Ask your CS 2050 questions here.

 sites.google.com/g.harvard.edu/cs-2050

 Readme

 Activity

 Custom properties

☆ 0 stars

👁 0 watching

🔗 0 forks

Releases

No releases published

[Create a new release](#)

THIS WEEK

Spring Recess

Week 8 - Extended Example: Training Large Language Models



Week 9 - Extended Example: Distributed Linear Algebra



Week 10 - Extended Example: Molecular Dynamics



Week 11 - Extra Topic: Kokkos



Week 12 - Project Work Week



Week 13 - Review



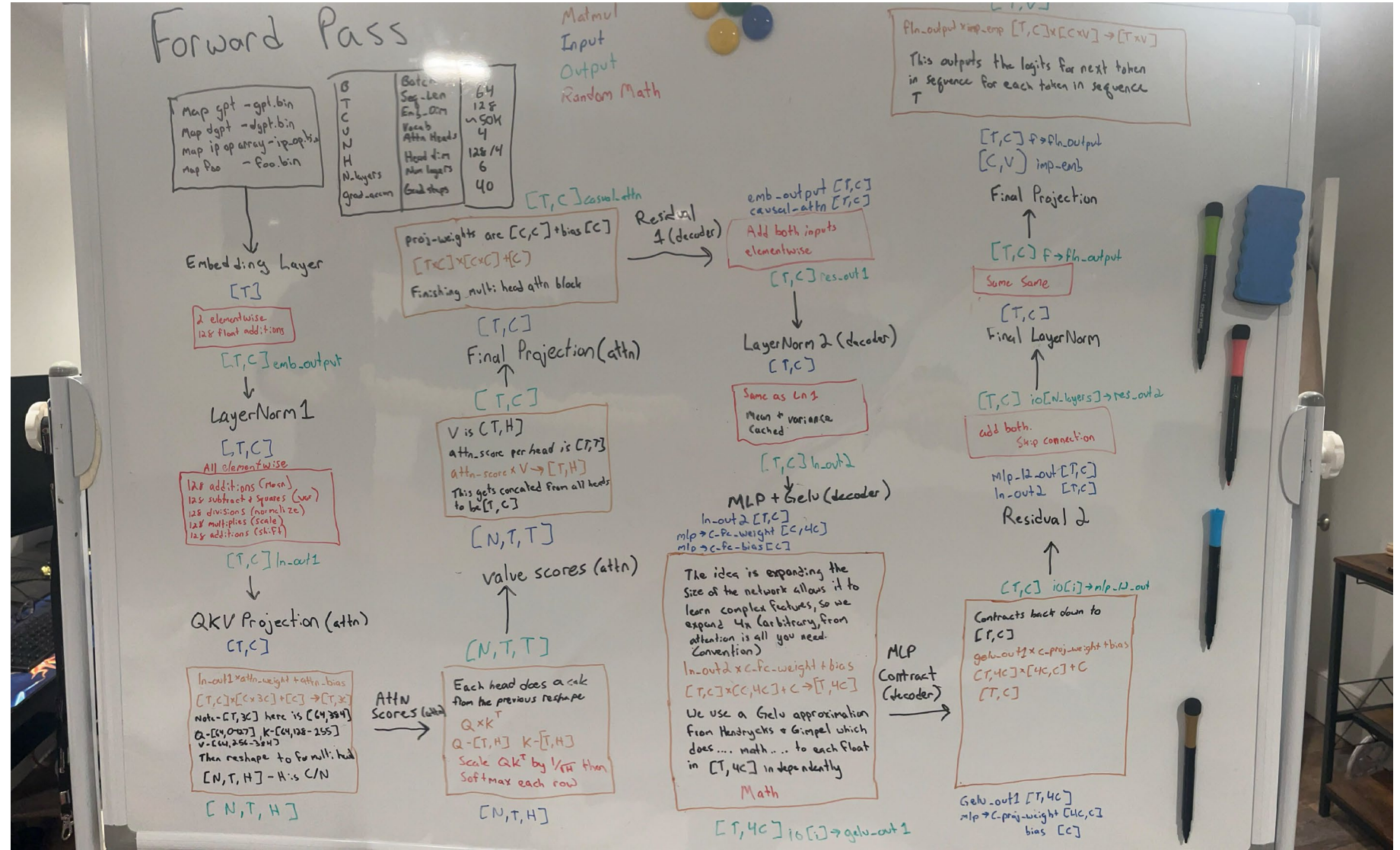


Manqing Liu



AJ Bailly

NANO GPT-2050



Loss Calculation / Backwards Pass

Calculate Loss

$[T, V]$

Red running out 11

Softmax over logits
divide by T
divide by grad-accum

$[T, V]$

Backpass all of P & Gs
model structure

Output Projection

$[T, V] \rightarrow \text{dfl-output}$
 $[V, C] \rightarrow \text{dnp-emb (orig)}$
 $[T, C] \rightarrow \text{dfln-output}$

2 matmuls here

1.) $\text{dfl-output} \times \text{inp-emb}$
 $[T, V] \times [V, C] \rightarrow [T, C] \rightarrow \text{dfln-output}$
"How much should final layerNorm change?"

* Intuition - dfl-output says
"you should be more or less conf
about this word being next", this
matmul projects that idea back
down to C dimension

2.) $\text{dfl-output} \times \text{dfln-output}$
 $[V, T] \times [T, C] \rightarrow [V, C]$

$[T, C] \rightarrow \text{dfln-output}$

$[V, C] \rightarrow \text{dnp-emb} \rightarrow \text{dnp-emb}$
this gets to be AdamV

Layer Norm Final
 $[T, C]$

Takes io[5] $\rightarrow$ res.out2 +
 dfln-output from above + does
... math ... elementwise

B	Batch
T	Seq-len
C	Emb-Dim
V	Vocab Size
N	Num Heads
H	Head Dim
N layers	Decoder Layers
Grad-accum	

LayerNorm2

$[T, C]$

4 matmuls, 1 elementwise mult

First 2 matmuls.)

Forward was $\text{mlp-l2-out} = \text{gelu-out} \times \text{C-proj}$
 $[T, C] = [T, 4C] \times [4C, C]$

1.) $\text{dgelu-out} = \text{dmlp-l2-out} \times \text{C-proj-weight}^T$
 $[T, 4C] = [T, C] \times [C, 4C]$

2.) $\text{dproj-weight} = \text{gelu-out}^T \times \text{dmlp-l2-out}$
 $[4C, C] = [4C, T] \times [T, C]$

d-proj-bias is column sum of

Elementwise mult of mlp grad by Gelu

2nd 2 matmuls.)

Forward was $\text{mlp-l2-out} = \text{ln-out2} \times \text{C-fe-weights}$
 $[T, 4C] [T, C] \times [C, 4C]$

1.) $\text{dln-out2} = \text{dmlp-l2-out} \times \text{C-fe-weight}^T$
 $[T, C] [T, 4C] \times [4C, C]$

2.) $\text{dfe-weight} = \text{ln-out2}^T \times \text{dmlp-l2-out}$
 $[T, 4C] [T, C] \times [C, 4C]$

d-fe-bias[5] is column sum of

written by elementwise step

MLP Backprop

see notes ... takes everything

$[T, C]$
 $[T, C]$ dres.out2

Residual 2

Duplicates gradient
one goes through MLP
one for skip

$[T, C]$

$[T, C]$ dres2-out
 $[T, C]$

Combines outputs
from res2 + mlp-bias
elementwise

$[T, C]$ dres.out1

Residual 1
 $[T, C]$

Same as res 2, split
the residual for skip
connection

$[T, C]$ datt-out
 $[T, C]$ dres.out2 + dres.out

Previous layer's
residual + skip
this layer's

Attention Backwards

8 matmuls
First 2.)

Forward was $\text{attn-out} = \text{cousl-attn} \times \text{proj-weight}$
 $[T, C] [T, C] \times [C, C]$

1.) $\text{dattn} = \text{datt-out} \times \text{proj-weight}^T$
 $[T, C] [T, C] \times [C, C]$

2.) $\text{dproj-weight} = \text{cousl-attn}^T \times \text{datt-out}$
 $[C, C] = [C, T] \times [T, C]$

Forward was $S = Q \times K^T$

3.) $\text{dS} = \text{dattn} \times V^T$ 4.) $\text{dK} = \text{dS}^T \times \text{dattn}$
 $[T, H] [T, H] \times [H, T]$ $[T, H] [T, H] \times [T, H]$

Manual calculation of dS-row due to
copying of dS here ... math

5.) $\text{dQ} = \text{dS-row} \times K / \sqrt{H}$ 6.) $\text{dK} = \text{dS-row}^T \times Q$
 $[T, H] [T, H] \times [H, H]$ $[T, H] [T, H] \times [T, H]$

Forward projection was $\text{ln-out1} \times \text{attn-weight} = \text{gelu}$
 $[T, C] [T, C] \times [C, C]$

7.) $\text{dln-out1} = \text{gelu} \times \text{attn-weight}^T$ concat dQ, dK
 $[T, C] [T, C] \times [C, C]$ $[T, H] [T, H] \times [T, H]$

8.) $\text{dattn-weight} = \text{ln-out1}^T \times \text{dln-out1}$ 4 heads
 $[C, C] [C, T] [T, H]$

Momentum - exponential moving average of
gradients
velocity - exponential avg of gradients squared
Bias correction
Adaptive step - $\hat{m}/\sqrt{s}$ - weights with high velocity
take small steps
Weight decay
No matmuls

AdamV

Gets L2 norm of every
gradient array + normalizes

Gradient Clipping

$[V, C]$ only T rows
update
 $[T, C]$

updates embedding
weights elementwise

$[T, C]$
 $[V, C]$
Embedding

Layer Norm 1

$[T, C]$ dmb.output

Sends signal to
next layer's residual
& normalizes dln-out1
writes signal for emb

EXAMPLE SETUP #1

- Pull the latest Lecture Examples
- Go to Example 4 and **sbatch** the Slurm script

EXAMPLE SETUP #2

- Launch VS Code

[Home](#) / [My Interactive Sessions](#) / [Code Server - COMPSCI 2050 \(CPU\)](#)

Interactive Apps

Desktops

 Linux Desktop on academic

Servers

 Code Server - COMPSCI 2050 (CPU)

 Code Server - COMPSCI 2050 (GPU)

 Code Server - Universal

Code Server - COMPSCI 2050 (CPU)

This app will launch a [VS Code](#) server using [Code Server](#).

Number of hours

Number of CPUs

[Launch](#)

* The Code Server - COMPSCI 2050 (CPU) session data for this session can be accessed under the [data root directory](#).

LARGE LANGUAGE MODELS

Agenda

01



History of LLMs

GPT-2 to modern frontier models

02



Training Phases

Pre-training, mid-training, and post-training

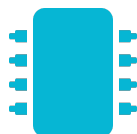
03



Transformer Architecture

Attention, FFN, forward & backward pass

04



NanoGPT & Parallelization

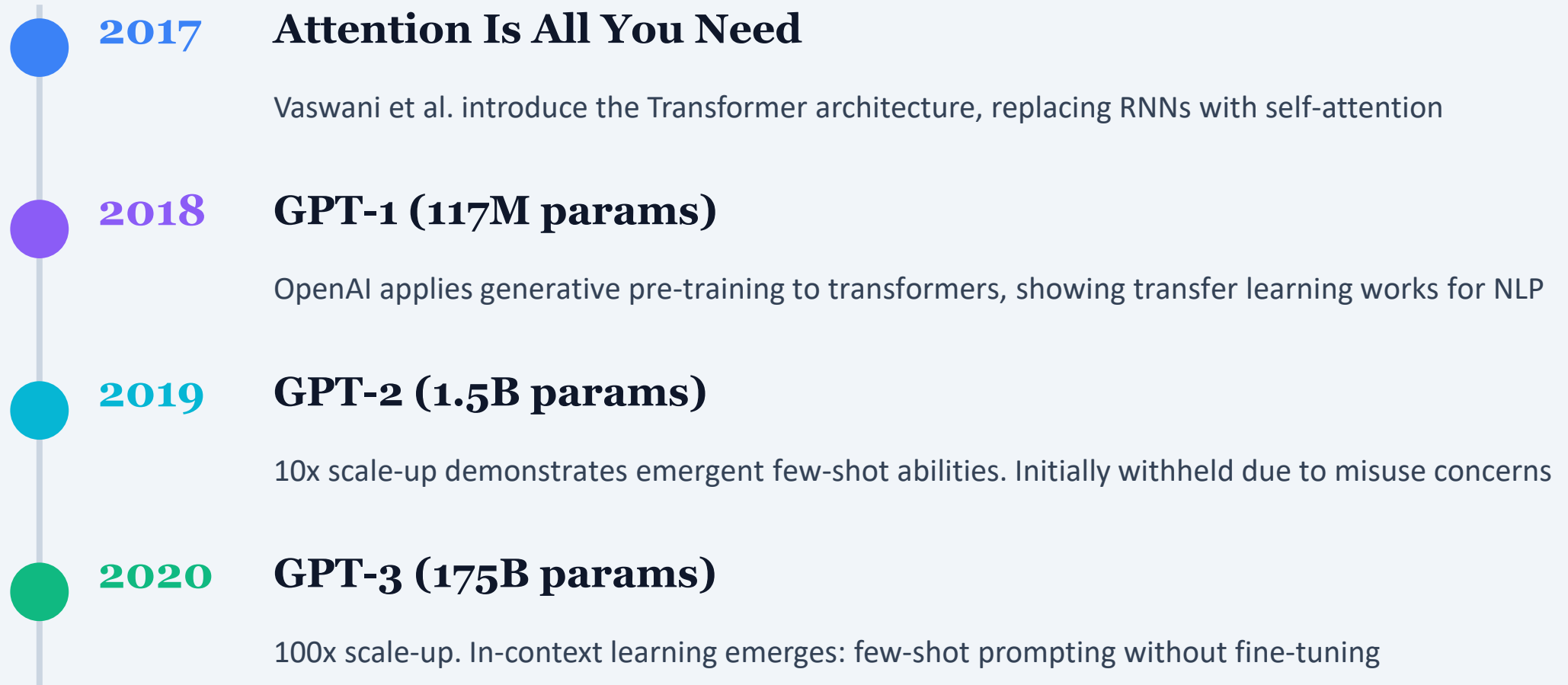
MPI, OpenMP, CUDA in our codebase

01

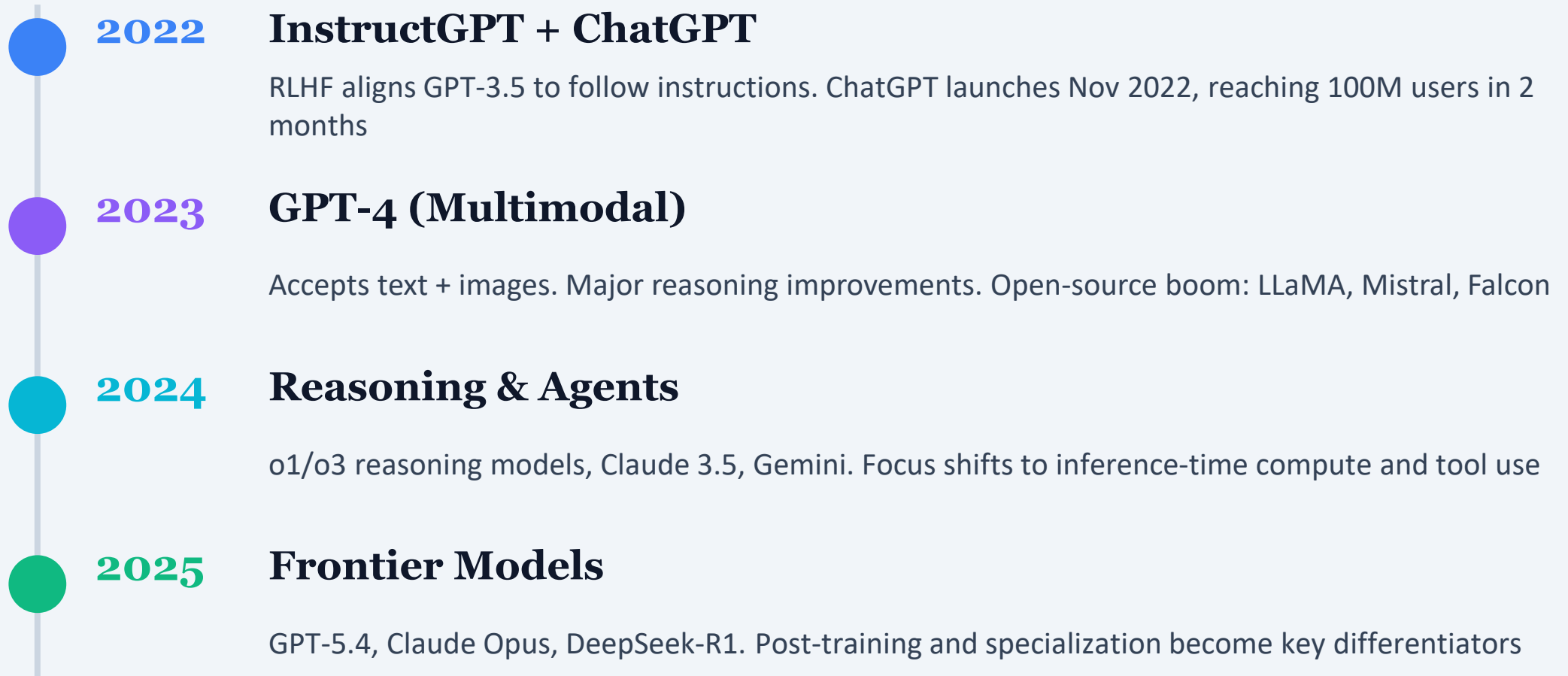
History of Large Language Models

From GPT-2 to the frontier model era

The Rise of Large Language Models



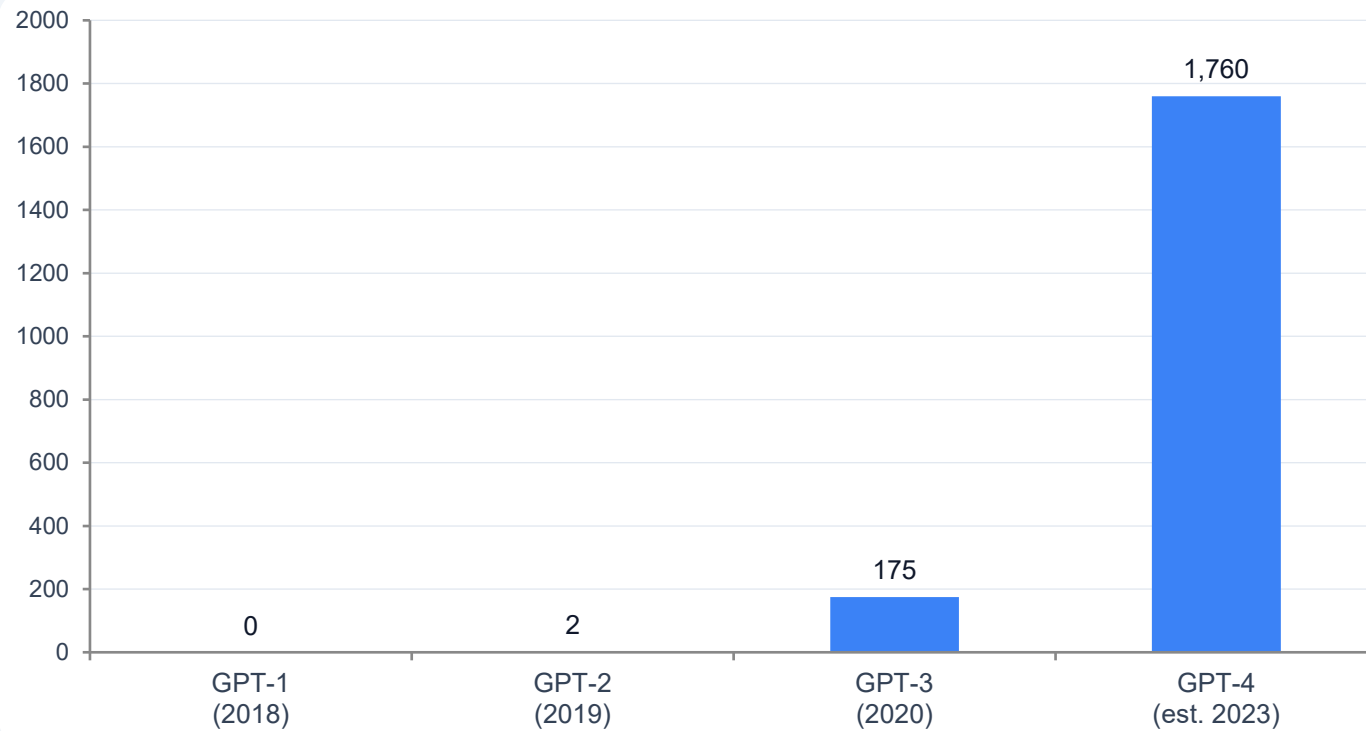
The ChatGPT Era and Beyond



The Scaling Hypothesis

What is a token? A token is a sub-word unit produced by BPE (Byte-Pair Encoding). For example, "unhappiness" becomes 3 tokens: ["un", "happi", "ness"]. GPT-2 uses a ~50k-token vocabulary. Models predict the next token during training.

Model Size Growth (Parameters in Billions)



Each generation scaled ~10-100× in parameters, driving qualitative jumps in capability.

Kaplan Scaling Laws

Loss decreases as a power law when you increase any of the three axes: parameters (model size), data (number of training tokens), and compute (FLOPs).

Chinchilla Optimal

DeepMind (2022) showed the optimal ratio is ~20 tokens per parameter. A 1B-param model needs ~20B training tokens to be compute-efficient.

Emergent Abilities

Capabilities like few-shot learning and chain-of-thought reasoning only appear once a model crosses a critical size threshold.

02

Phases of LLM Training

Pre-training, mid-training, and post-training

The LLM Training Pipeline

Pre-Training

Learn language

- Next-token prediction on massive corpora
- Trillions of tokens from web, books, code
- Bulk of compute budget (months on 1000s of GPUs)
- Result: base model with broad knowledge



Mid-Training

Refine knowledge

- Continued training on curated data
- Domain adaptation (code, math, science)
- Long-context extension training
- Knowledge cutoff updates



Post-Training

Align & specialize

- SFT: supervised fine-tuning on instructions
- RLHF: reinforcement learning from human feedback
- DPO/RLAIF: preference optimization variants
- Result: helpful, harmless, honest assistant

Pre-Training: Next-Token Prediction

The Core Objective

Given a sequence of tokens, predict the next token. This simple objective, scaled up massively, produces remarkably capable language models.

Cross-Entropy Loss:

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \log P(w_t \mid w_1, w_2, \dots, w_{t-1})$$

What the variables mean:

- T : The total number of words (tokens) in the sequence.
- t : The current time step or position in the sequence.
- w_t : The target word to predict at step t .
- $P(w_t \mid w_1, \dots, w_{t-1})$: The predicted probability of the correct next word, given all the preceding words.

NanoGPT Config

Layers	6
Heads	4
d_model	128
Seq Length	64
Batch Size	4
Vocab Size	~50,000
Optimizer	AdamW
LR Schedule	Cosine + warmup

Training Data: The Shakespeare Corpus

NanoGPT trains on the complete works of Shakespeare, tokenized with GPT-2's BPE encoder

Data Pipeline

1

Raw Text

data/input.txt — Complete Shakespeare from Project Gutenberg. ~5.5 MB, ~124k lines. Gutenberg boilerplate is stripped by the prep script.

2

BPE Tokenization

python data/prepare_bpe.py runs GPT-2's tiktoken encoder. Each word is split into sub-word tokens from a ~50k vocabulary. "unhappiness" → ["un", "happi", "ness"].

3

Token IDs

data/output.txt — ~1.85M space-separated integer IDs, ready for the C++ trainer. Also produces vocab_tokens.bin for inference decoding.

Corpus Statistics

Raw size	~5.5 MB
Characters	~5.46M
Lines	~124,000
BPE tokens	~1.85M
Unique tokens used	~15,000 of 50k vocab

Sample: Raw → Tokens

"To be, or not to be"

↓ BPE encode ↓

[1675, 307, 11, 393, 407, 284, 307]

03

The Transformer Architecture

"Attention Is All You Need" — Vaswani et al., 2017

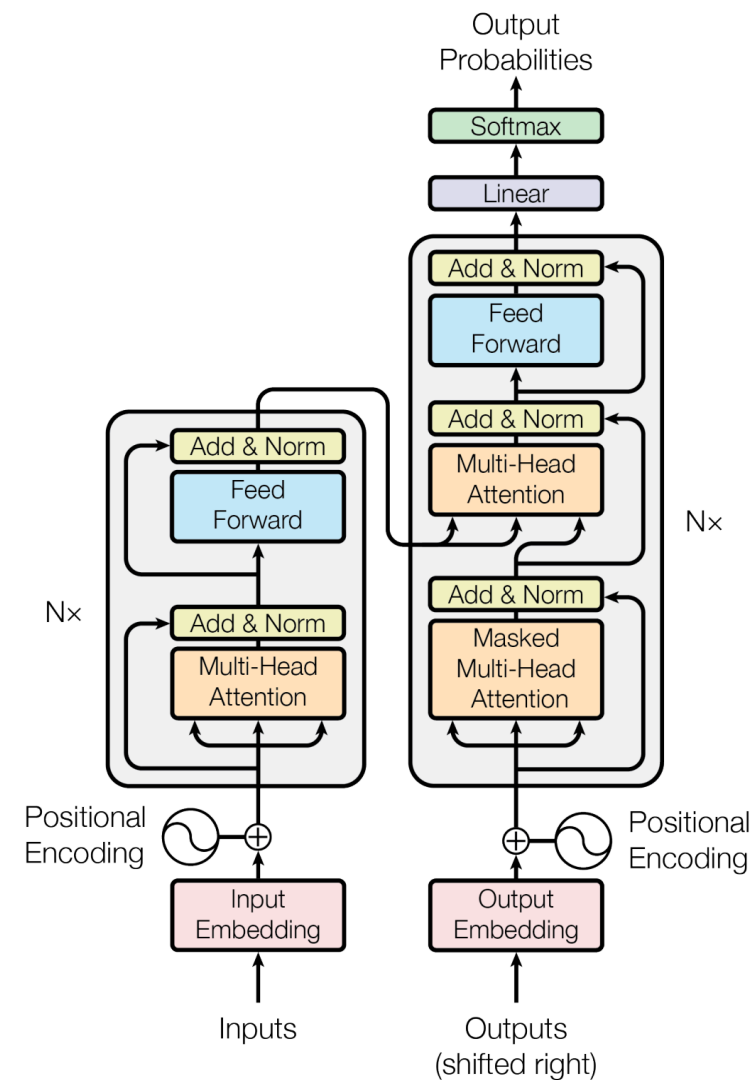


Figure 1: The Transformer - model architecture.

Why Transformers Dominate

Before transformers, RNNs and LSTMs were the standard for sequence modeling. Here's why transformers replaced them.

Property	RNN / LSTM	CNN (ConvNets)	Transformer
Parallelism	Sequential; each token waits for the previous hidden state	Parallel within window, but limited receptive field	Fully parallel; all tokens processed simultaneously
Long-range context	Vanishing gradients lose information over ~100 tokens	Fixed window size (e.g. 5–15 tokens) limits context	Full sequence attention: every token sees every other token
Training speed	Slow; cannot utilize GPU parallelism well	Faster than RNN, but stacking layers is expensive	Fast; matmul-heavy, perfectly suited for GPU/TPU
Scalability	Hard to scale beyond ~1B parameters	Limited by depth/width trade-offs	Scales predictably to trillions of parameters (scaling laws)
Key weakness	Cannot look ahead; forgets distant past	Cannot capture dependencies beyond window	$O(T^2)$ attention cost; needs positional encoding

The Core Advantage

Self-attention replaces sequential recurrence with a single parallelizable matrix operation ($Q \cdot K^T$). This means training time scales with hardware, not sequence length

Transformer: The Core Math (1/2)

Vaswani et al., 2017 — "Attention Is All You Need"

0 Input Embedding

Translates discrete text tokens into continuous vector spaces.

- **Process:** Each token in a sequence of length L is mapped to a vector of dimension d_{model} .
- **Dimensions:** $X_{embed} \in \mathbb{R}^{L \times d_{model}}$ (In the original paper, $d_{model} = 512$).

1 Positional Encoding

Injects information about the relative or absolute position of the tokens, as the attention mechanism itself has no sense of sequence order.

- **Formulas:**

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

- **Dimensions:** Added directly to the embeddings. $X_{input} = X_{embed} + PE \in \mathbb{R}^{L \times d_{model}}$.

2 Scaled Dot-Product Attention

The core mechanism allowing the model to weigh the relevance of different words in the sequence.

- **Formula:**

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- **Dimensions:** Queries Q , Keys $K \in \mathbb{R}^{L \times d_k}$, and Values $V \in \mathbb{R}^{L \times d_v}$. Output dimension is $\mathbb{R}^{L \times d_v}$.

Transformer: The Core Math (2/2)

3 Multi-Head Attention

Instead of performing a single attention function, the model linearly projects the queries, keys, and values h times into smaller sub-spaces, allowing it to attend to different types of relationships simultaneously.

- **Formulas:**

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{where } \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

- **Dimensions:** Projection matrices $W_i^Q, W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$, and output weight $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$. The paper uses $h = 8$ heads, and $d_k = d_v = d_{\text{model}}/h = 64$.

4 Layer Normalization

Used inside the "Add & Norm" blocks. Instead of normalizing across the batch, it normalizes the activations across the feature dimension (the embedding size) for each token independently.

- **Formula:**

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \gamma + \beta$$

(Where μ is the mean, σ^2 is the variance, ϵ is a tiny number to prevent division by zero, and $\odot$ represents element-wise multiplication).

- **Dimensions:** Input $x \in \mathbb{R}^{d_{\text{model}}}$. The mean μ and variance σ^2 are scalars calculated per token. The learnable parameters (scale γ and shift β) are vectors of dimension $\mathbb{R}^{d_{\text{model}}}$. The final output remains $\mathbb{R}^{L \times d_{\text{model}}}$.

5 Feed-Forward Network (FFN)

Applied identically but independently to each position. It consists of two linear transformations with a ReLU activation in between.

- **Formula:**

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- **Dimensions:** $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ and $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$. The paper uses an inner dimension of $d_{\text{ff}} = 2048$. Output is $\mathbb{R}^{L \times d_{\text{model}}}$.

6 Output: Softmax over Vocabulary

Converts the final decoder output into a probability distribution over the entire vocabulary to predict the next word.

- **Formula:**

$$P(\text{next word}) = \text{softmax}(X_{\text{final}} W_{\text{vocab}})$$

- **Dimensions:** Final hidden state $X_{\text{final}} \in \mathbb{R}^{L \times d_{\text{model}}}$, Vocabulary Weight Matrix $W_{\text{vocab}} \in \mathbb{R}^{d_{\text{model}} \times |V|}$ (where $|V|$ is the vocabulary size). The resulting output matrix is $\mathbb{R}^{L \times |V|}$.

Complete Forward Pass with Dimensions

1 Embed
 $x = W_{\text{token}}[x] + W_{\text{position}}[\text{pos}]$
[B, T] → [256, 128]

2 LayerNorm
 $\hat{x} = \gamma(x - \mu) / \sqrt{(\sigma^2 + \epsilon)} + \beta$
[256, 128] → [256, 128]

3 Attention
 $\text{attn_out} = \text{MHA}(\text{LayerNorm}(x))$
[256, 128] → [256, 128]

4 Residual
 $x = x + \text{attn_out}$
[256, 128] + [256, 128]

5 LayerNorm
 $\hat{x} = \gamma(x - \mu) / \sqrt{(\sigma^2 + \epsilon)} + \beta$
[256, 128] → [256, 128]

6 FFN
 $\text{ffn_out} = \text{FFN}(\text{LayerNorm}(x))$
128 → 512 → 128

7 Residual
 $x = x + \text{ffn_out}$
[256, 128] + [256, 128]

×6 Repeat
Steps 2-7 for each of L=6 layers
 $x_{\text{final}} = \text{embedding} + \text{attn}_1 + \text{ffn}_1 + \dots + \text{attn}_6 + \text{ffn}_6$

8 Logits
 $\text{Logits} = x_{\text{final}} \cdot W_{\text{token}}^T$ (weight-tied)
[256, 128] → [256, 50k]

9 Loss
 $\text{CE} = -\log(\text{softmax}(\text{logits}))$
scalar

Backward Pass: Gradient Dimensions

Every gradient has the same shape as its corresponding activation or weight matrix

Loss → dLogits

$$dL = \text{softmax}(\text{logits}) - \text{one_hot}(\text{target})$$

$$[B \cdot T, V] = [256, 50k]$$

Scaled by $1/(B \cdot T)$. This is the starting gradient.

dLogits → dX, dW_token

$$dX = dL \cdot W_{\text{token}} \quad dW_{\text{token}} += X^T \cdot dL$$

$$dX: [256, 128] \quad dW: [50k, 128]$$

Backprop through weight-tied output projection.

Layer loop (6→1)

$$d\text{FFN} \rightarrow d\text{LN2} \rightarrow +d\text{Res} \rightarrow d\text{Attn} \rightarrow d\text{LN1} \rightarrow +d\text{Res}$$

$$\text{all gradients } [256, 128]$$

Reverse order. Residual: dRes passes gradient straight through.

Attn backprop

$$\begin{aligned} dQ &= d\text{Scores} \cdot K \\ dK &= d\text{Scores}^T \cdot Q \\ dV &= \text{Attn}^T \cdot d\text{Out} \end{aligned}$$

$$dQ, dK, dV: [T, H] = [64, 32]$$

dEmbeddings

$$dW_{\text{token}}[x_t] += dX[t] \quad (\text{scatter-add})$$

$$dW_{\text{tok}}: [50k, 128] \quad dW_{\text{pos}}: [64, 128]$$

Accumulate gradients by token/position index.

Key Takeaways



LLMs evolved rapidly

From GPT-2 (1.5B, 2019) to trillion-parameter frontier models in just 6 years, driven by scaling laws and architectural innovation.



Training has three phases

Pre-training (next-token prediction at scale), mid-training (domain adaptation), and post-training (RLHF alignment) each serve distinct purposes.



Attention is the key insight

Self-attention replaces recurrence with parallel pairwise interactions, enabling both better modeling of long-range dependencies and GPU-friendly computation.



Parallelism is essential

NanoGPT demonstrates three complementary strategies: MPI for multi-node data parallelism, OpenMP for intra-node threading, and CUDA for GPU kernel acceleration.

EXAMPLE 1

SHAKESPEARE CORPUS

EXAMPLE 2

FIRST LOOK AT NANOGPT-2050

04

Parallelization

MPI, OpenMP, and CUDA strategies

Three Levels of Parallelism

OpenMP Thread-Parallel

Intra-Node

Shared-memory parallelism within a node. Loops over (batch, head) pairs use a combination of collapse, scheduling(dynamic) and parallel fors. Thread-private buffers for reductions.

```
OMP_NUM_THREADS=[1-24]
```

MPI Data-Parallel

Multi-Node

Each rank trains on different mini-batches. Gradients synchronized via MPI_Allreduce(SUM) after accumulation. All ranks maintain identical weights.

```
gpt.cpp
```

CUDA GPU Kernels

Device-Level

Custom kernels for LayerNorm, Softmax, GELU, embeddings. cuBLAS for all matmuls. Single flat cudaMalloc for all activations. Weights are GPU-resident.

```
gpu_ops.cu + cuda_ai.cpp
```

MPI: Data-Parallel Training

Rank 0	Rank 1	Rank 2	Rank 3
• Sample batch • Forward pass • Backward pass • Accumulate grads	• Sample batch • Forward pass • Backward pass • Accumulate grads	• Sample batch • Forward pass • Backward pass • Accumulate grads	• Sample batch • Forward pass • Backward pass • Accumulate grads

`MPI_Allreduce(MPI_SUM)` — Gradient Synchronization

Gradient Accumulation

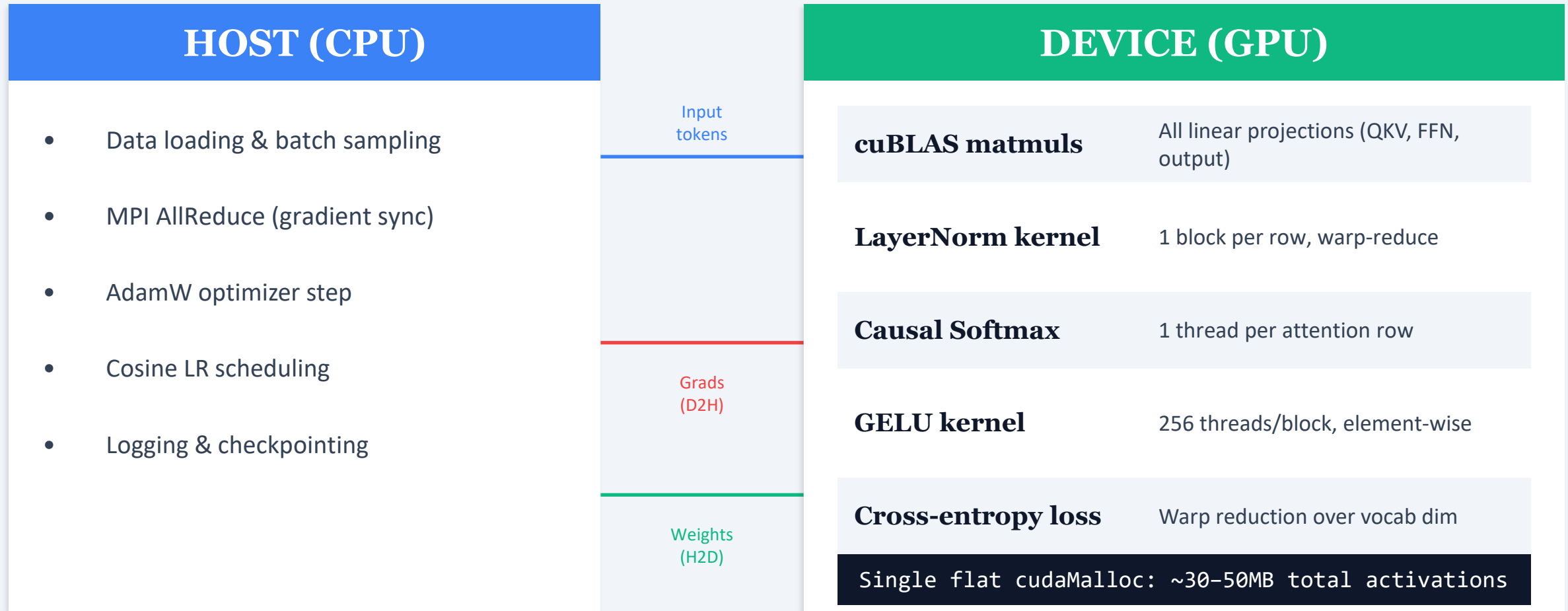
Each rank runs `local_accum = grad_accum_steps / world_size` micro-batches before AllReduce. Gradients scaled by `1/grad_accum_steps`. This minimizes communication rounds.

Communication Cost

With gradient accumulation (40 steps), compute per rank (200ms) >> AllReduce time (20ms). Communication is amortized and NOT the bottleneck at this scale.

CUDA: GPU-Accelerated Training

Data Flow: Host ↔ Device



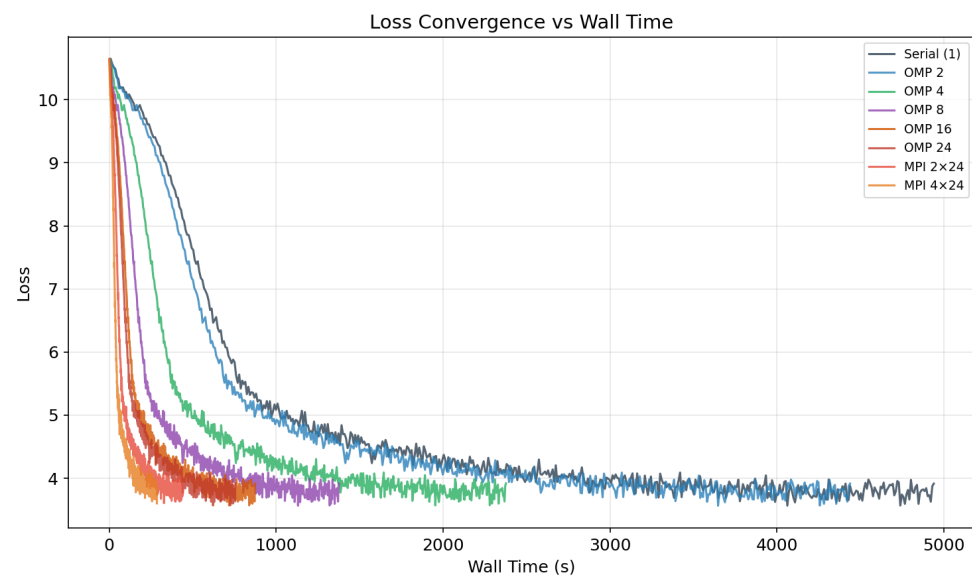
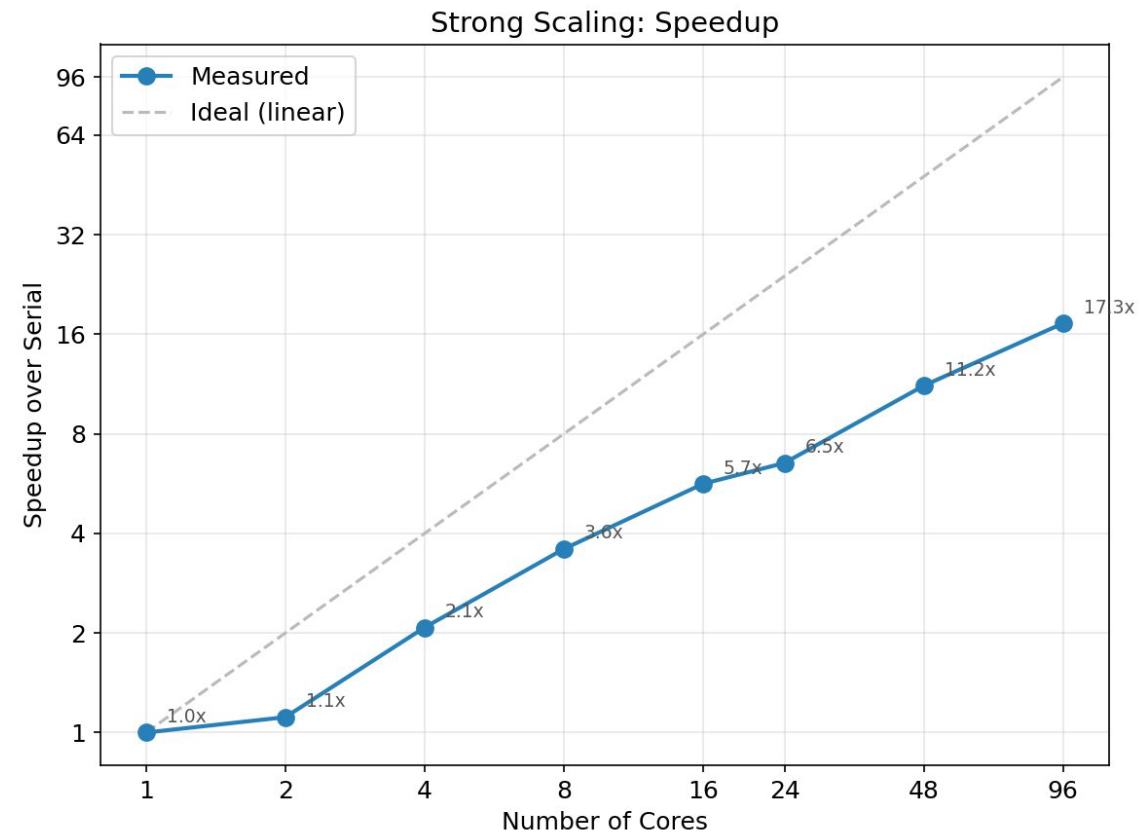
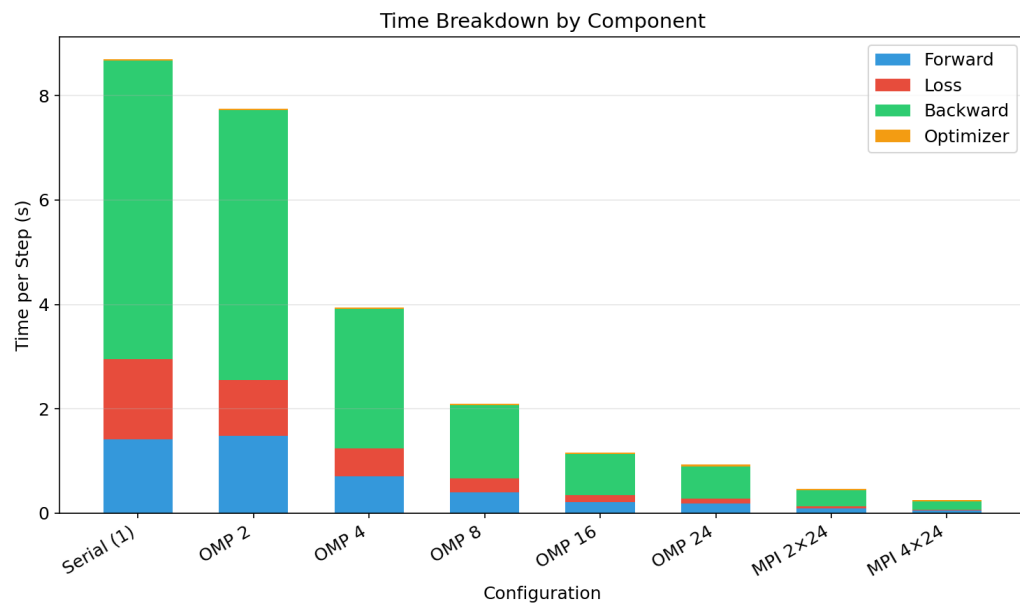
Where Does Parallelization Help Most?

Component	CPU (OpenMP)	GPU (CUDA)	MPI
Embedding	collapse(2) on B×T	256 threads/block	—
Attention $Q \cdot K^T$	dynamic per (B,H)	cuBLAS per head	—
Softmax	collapse(3)	1 thread/row	—
LayerNorm	parallel mean/var	warp-reduce	—
MLP matmuls	BLAS calls	cuBLAS	—
Gradient Sync	—	—	AllReduce(SUM)
Inference	Limited	Single fwd fast	Not used



Key Insight

MLP matmuls dominate compute (~2/3 of FLOPs per layer) and benefit most from GPU acceleration. Inference is inherently sequential (autoregressive) and cannot parallelize across token positions, though individual forward passes are still faster on GPU.



- Sub- Linear increase from OpenMP Parallelization
 - Model size is the limiting factor
- Near Linear increase from MPI implementation
 - Slight overhead from All-Reduce

Full MLP FLOPs & GPU Payoff

Both MLP matmuls combined, then scaled across all 6 layers

MLP FLOPs per Layer

Operation ($Y=X \cdot W+b$)	Shapes	Calculation	FLOPs
Expand: $X \cdot W_1 + b_1$	$[256, 128] \times [128, 512]$	$2 \times 256 \times 128 \times 512$	33.6M
Bias add	256×512	131,072	0.1M
GELU activation	~10 ops/elem	$10 \times 256 \times 512$	1.3M
Compress: $H \cdot W_2 + b_2$	$[256, 512] \times [512, 128]$	$2 \times 256 \times 512 \times 128$	33.6M
Bias add	256×128	32,768	0.03M
MLP Total per Layer		≈ 68.6 MFLOPs	

Full Model Summary

MLP $\times$ 6 layers	411.6M FLOPs
Attention $\times$ 6 layers	≈ 210M FLOPs
Embeddings + LN	≈ 15M FLOPs
Total forward pass	≈ 637M FLOPs

$\sim 65\%$

of all FLOPs are MLP matmuls
GPU cuBLAS: $\sim 100\times$ faster than CPU

EXAMPLE 3

NANOGPT-2050 WITH OPENMP

EXAMPLE 4

BEYOND TOY EXAMPLES